

LTE Standard(U)系列 QuecOpen 文件系统 API 参考手册

LTE Standard 模块系列

版本：1.1

日期：2023-08-28

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更描述
-	2020-10-21	Kevin WANG	文档创建
1.0	2021-08-03	Kevin WANG/ Herry GENG	受控版本
1.1	2023-08-28	Kevin WANG/ Herry GENG	1. 新增适用模块 EC200D 系列、EC200G-CN、EC600G 系列、EC800G-CN、EG700G-CN、EG800G 系列、EG912U-GL 和 EG915U 系列。 2. 新增 EC200D 系列模块文件路径备注(第 1.1 章)。 3. 更新文件系统分区 (第 2.2 章和 3.3.1 章)。

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
1.1. 适用模块	7
2 文件系统简介	9
2.1. 文件系统设计原则	9
2.2. 文件系统分区	9
2.3. 文件系统功能简述	9
3 文件系统 API	10
3.1. 头文件	10
3.2. 函数概览	10
3.3. 函数详解	12
3.3.1. ql_fopen	12
3.3.2. ql_close	12
3.3.3. ql_remove	13
3.3.4. ql_fread	13
3.3.5. ql_fwrite	14
3.3.6. ql_fseek	14
3.3.7. ql_frewind	15
3.3.8. ql_ftell	15
3.3.9. ql_fstat	16
3.3.10. ql_stat	16
3.3.11. ql_ftruncate	17
3.3.12. ql_fsize	17
3.3.13. ql_file_exist	18
3.3.14. ql_mkdir	18
3.3.15. ql_opendir	18
3.3.16. ql_closedir	19
3.3.17. ql_readdir	19
3.3.17.1. qdirent	20
3.3.18. ql_telldir	20
3.3.19. ql_seekdir	21
3.3.20. ql_rewinddir	21
3.3.21. ql_rmdir	22
3.3.22. ql_rename	22
3.3.23. ql_fs_free_size	22
3.3.24. ql_nvm_fwrite	23
3.3.25. ql_nvm_fread	24
3.3.26. ql_sfs_setkey	24

3.3.27.	ql_fs_free_size_by_fd	25
3.3.28.	ql_fputc	25
3.3.29.	ql_fgetc	26
3.3.30.	ql_ferror	26
3.3.31.	ql_fsync	27
3.3.32.	ql_cust_nvm_fwrite	27
3.3.33.	ql_cust_nvm_fread	28
4	示例	29
5	附录 参考文档及术语缩写	30

表格索引

表 1: 适用模块	7
表 2: 函数概览	10
表 3: 参考文档	30
表 4: 术语缩写	30

1 引言

移远通信 LTE Standard(U)系列模块支持 QuecOpen®方案。QuecOpen®是基于 RTOS 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]和[2]。

本文档主要介绍在 QuecOpen®方案下，LTE Standard(U)系列模块的文件系统、文件系统 API 及相关示例。

1.1. 适用模块

表 1: 适用模块

模块系列	模块
LTE Standard(U)	EC200D 系列
	EC200G-CN
	EC600G 系列
	EC800G-CN
	EC200U 系列
	EC600U 系列
	EG700G-CN
	EG800G 系列
	EG500U-CN
	EG700U-CN
	EG912U-GL
	EG915U 系列

备注

对于 EC200D 系列模块，文中涉及到的文件路径，路径名称中均不包含 *components*。

2 文件系统简介

2.1. 文件系统设计原则

LTE Standard(U)系列 QuecOpen 模块文件系统 API 设计尽量靠近 C 标准函数库。模块内部文件系统采用 SFFS 格式，因此不兼容 PC 或 Linux 上的任何文件系统。

2.2. 文件系统分区

文件系统为用户提供三个磁盘分区，盘符分别为 UFS:、EFS:、EXNSFFS:、EXNAND:和 SD:。

- UFS 盘位于内置 Flash，用于存放用户的普通文件（该分区内还设置 SFS 加密文件目录，用于存放用户加密文件）。
- EXNSFFS 盘为外部 4 线 SPI NOR Flash 文件系统分区，需要用户外挂 4 线 SPI NOR Flash 才可使用。
- EXNAND 盘为外部 SPI NAND FLASH 文件系统分区，需要用户外挂 SPI NAND Flash 才可使用。
- EFS 盘为外部 6 线 SPI NOR Flash 文件系统分区，需要用户外挂 6 线 SPI NOR Flash 才可使用。
- SD 盘为 SD 卡分区。

备注

对于 EC200D 系列模块，暂时不支持 EXNAND、EFS 和 EXNSFFS 分区。

2.3. 文件系统功能简述

LTE Standard(U)系列 QuecOpen 模块的文件系统可进行以下操作：

- 文件操作：打开、读取、写入、关闭、删除和重命名文件、文件指针跳转、获取当前文件指针位置、设置文件指针回到开始、改变和获取文件大小等。
- 目录操作：创建、打开、读取和关闭目录、删除空目录、目录指针跳转、获取当前目录指针位置和设置目录指针回到开始等。
- 其他功能：获取文件状态和分区剩余空间大小、写入和读取配置文件、配置 SFS 文件密钥、写入和读取一个字符、同步已修改的数据到存储设备等。

3 文件系统 API

3.1. 头文件

文件系统 API 头文件为 `ql_fs.h`，位于 SDK 包的 `components\ql-kernel\inc` 目录下。若无特别说明，本文档所涉头文件均在该目录下。

3.2. 函数概览

表 2：函数概览

函数	说明
<code>ql_fopen()</code>	打开指定文件并返回文件句柄
<code>ql_fclose()</code>	关闭指定文件
<code>ql_remove()</code>	删除指定文件
<code>ql_fread()</code>	从指定文件中读取数据
<code>ql_fwrite()</code>	向指定文件中写入数据
<code>ql_fseek()</code>	设置文件指针的位置
<code>ql_frewind()</code>	设置文件指针于文件开头
<code>ql_ftell()</code>	获取文件指针相对于文件开头的偏移值
<code>ql_fstat()</code>	通过文件句柄获取文件状态
<code>ql_stat()</code>	通过文件名获取文件状态
<code>ql_ftruncate()</code>	从文件开头截取文件为指定长度
<code>ql_fsize()</code>	获取文件大小
<code>ql_file_exist()</code>	判断文件是否存在

<code>ql_mkdir()</code>	创建目录
<code>ql_opendir()</code>	打开指定目录并返回目录句柄
<code>ql_closedir()</code>	关闭指定目录
<code>ql_readdir()</code>	从指定目录中获取文件信息
<code>ql_telldir()</code>	获取目录流位置
<code>ql_seekdir()</code>	设置目录流读取位置
<code>ql_rewinddir()</code>	设置目录流读取位置于目录开头
<code>ql_rmdir()</code>	删除指定空目录
<code>ql_rename()</code>	重命名文件
<code>ql_fs_free_size()</code>	通过文件（带路径）、目录路径或分区名，获取其所在分区的剩余空间大小
<code>ql_nvm_fwrite()</code>	写入简单配置文件
<code>ql_nvm_fread()</code>	读取简单配置文件
<code>ql_sfs_setkey()</code>	配置读写的 SFS 文件密钥
<code>ql_fs_free_size_by_fd()</code>	通过文件句柄获取文件所在分区的剩余空间大小
<code>ql_fputc()</code>	向文件中写入一个字符
<code>ql_fgetc()</code>	从文件中读取一个字符
<code>ql_ferror()</code>	判断当前文件句柄是否有效
<code>ql_fsync()</code>	同步内存中已修改的文件数据到储存设备
<code>ql_cust_nvm_fwrite()</code>	写入用户自定义简单配置文件
<code>ql_cust_nvm_fread()</code>	读取用户自定义简单配置文件

3.3. 函数详解

3.3.1. ql_fopen

该函数用于打开指定文件并返回文件句柄。文件句柄为 32 位有符号型数。

- 函数原型

```
QFILE ql_fopen(const char * file_name, const char *mode)
```

- 参数

file_name:

[In] 文件名（带路径）。长度应不大于 132 字节，标准格式为“磁盘名:/路径名/文件名”。暂支持如下分区：

UFS	主分区，用于存放用户的普通文件
SFS	加密文件目录，与 UFS 共享一个分区，用于存放用户加密文件
EFS	外部 6 线 SPI NOR Flash 文件系统分区
SD	SD 卡分区
EXNSFFS	外部 4 线 SPI NOR Flash 文件系统分区
EXNAND	外部 SPI NAND FLASH 文件系统分区

mode:

[In] 文件打开方式。与 C 标准函数库一致，常用如“wb+”。

- 返回值

文件句柄 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.2. ql_close

该函数用于关闭指定文件。

- 函数原型

```
int ql_fclose(QFILE fd)
```

- 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

- 返回值

详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.3. ql_remove

该函数用于删除指定文件。

- 函数原型

```
int ql_remove(const char *file_name)
```

- 参数

file_name:

[In] 文件名称。格式与 `ql_fopen()` 中的 *file_name* 一致。

- 返回值

详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.4. ql_fread

该函数用于从指定文件中读取数据。

- 函数原型

```
int ql_fread(void *buffer, size_t size, size_t num, QFILE fd)
```

- 参数

buffer:

[In] 存储读取数据的内存地址。不可为空指针。

size:

[In] 需读取的每个元素大小。单位：字节。

num:

[In] 需读取的元素个数。最终读取数据大小为 $size \times num$ 。

fd:

[In] 文件句柄。即 `ql_fopen()` 执行成功后的返回值。

- 返回值

读取数据大小 函数执行成功
其他值 函数执行失败；详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.5. ql_fwrite

该函数用于向指定文件中写入数据。

- 函数原型

```
int ql_fwrite(void * buffer, size_t size, size_t num, QFILE fd)
```

- 参数

buffer:

[In] 存储写入数据的内存地址。不可为空指针。

size:

[In] 需写入的每个元素大小。单位：字节。

num:

[In] 需写入的元素个数。最终写入数据大小为 $\text{size} \times \text{num}$ 。

fd:

[In] 文件句柄。即 `ql_fopen()` 执行成功后的返回值。

- 返回值

写入数据大小 函数执行成功
其他值 函数执行失败；详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.6. ql_fseek

该函数用于设置文件指针的位置。

- 函数原型

```
int ql_fseek(QFILE fd, long offset, int origin)
```

- 参数

fd:

[In] 文件句柄。即 `ql_fopen()` 执行成功后的返回值。

offset:

[In] 文件指针位置偏移值。单位：字节。

origin:

[In] 开始添加 *offset* 的位置。包含如下值：

- QL_SEEK_SET* 文件开头
- QL_SEEK_CUR* 文件指针的当前位置
- QL_SEEK_END* 文件末尾

● 返回值

文件指针相对于文件开头的偏移值 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.7. ql_frewind

该函数用于设置文件指针于文件开头。

● 函数原型

```
int ql_frewind(QFILE fd)
```

● 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

● 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.8. ql_ftell

该函数用于获取文件指针相对于文件开头的偏移值。

● 函数原型

```
int ql_ftell(QFILE fd)
```

● 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

- 返回值

文件指针偏移值 函数执行成功

其他值 函数执行失败；详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.9. ql_fstat

该函数用于通过文件句柄获取文件状态。

- 函数原型

```
int ql_fstat(QFILE fd, struct stat *st)
```

- 参数

fd:

[In] 文件句柄。即 `ql_fopen()` 执行成功后的返回值。

st:

[Out] 文件状态结构体。不可为空指针。可参考 C 标准函数库中 `stat` 结构体。

- 返回值

详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.10. ql_stat

该函数用于通过文件名获取文件状态。

- 函数原型

```
int ql_stat(const char *file_name, struct stat *st)
```

- 参数

file_name:

[In] 文件名（带路径）。格式与 `ql_fopen()` 中的 `file_name` 一致。

st:

[Out] 文件状态结构体。不可为空指针。可参考 C 标准函数库中 `stat` 结构体。

- 返回值

详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.11. ql_ftruncate

该函数用于从文件开头截取文件为指定长度。

- 函数原型

```
int ql_ftruncate(QFILE fd, uint length)
```

- 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

length:

[In] 指定截取长度。单位：字节。

- 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.12. ql_fsize

该函数用于获取文件大小。单位：字节。

- 函数原型

```
int ql_fsize(int fd)
```

- 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

- 返回值

文件大小 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.13. ql_file_exist

该函数用于判断文件是否存在。

- 函数原型

```
int ql_file_exist(const char *file_path)
```

- 参数

file_path:

[In] 文件名（带路径）。格式与 *ql_fopen()* 中的 *file_name* 一致。

- 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.14. ql_mkdir

该函数用于创建目录。

- 函数原型

```
int ql_mkdir(const char *path, uint mode)
```

- 参数

path:

[In] 目录名（带路径）。路径格式与 *ql_fopen()* 中的 *file_name* 一致。

mode:

[In] 该参数为无效参数。此处为靠近 C 标准函数库而设计，输入 0 或正数即可。

- 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.15. ql_opendir

该函数用于打开指定目录并返回目录句柄。

- 函数原型

```
QDIR *ql_opendir(const char *path)
```

- 参数

path:

[In] 目录名（带路径）。路径格式与 *ql_fopen()* 中的 *file_name* 一致。

- 返回值

目录句柄 函数执行成功

NULL 函数执行失败

3.3.16. ql_closedir

该函数用于关闭指定目录。

- 函数原型

```
int ql_closedir(QDIR *pdir)
```

- 参数

pdir:

[In] 目录句柄。即 *ql_opendir()* 执行成功后的返回值。

- 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.17. ql_readdir

该函数用于从指定目录中获取文件信息。文件信息结构体 *qdirent* 详见第 3.3.17.1 章。

- 函数原型

```
qdirent * ql_readdir(QDIR *pdir)
```

- 参数

pdir:

[In] 目录句柄。即 *ql_opendir()* 执行成功后的返回值。

- 返回值

指向 *qdirent* 结构体的非空指针 函数执行成功

NULL 函数执行失败

3.3.17.1. qdirent

文件信息 *qdirent* 结构体定义如下：

```
typedef struct
{
    int d_ino;
    uint8 d_type;
    char d_name[256];
}qdirent
```

- 参数

类型	参数	描述
int	<i>d_ino</i>	索引节点号
uint8	<i>d_type</i>	文件类型
char	<i>d_name</i>	文件名，最长 255 字符

3.3.18. ql_telldir

该函数用于获取目录流位置。

- 函数原型

```
int ql_telldir(QDIR *pdir)
```

- 参数

pdir:

[In] 目录句柄。即 *ql_opendir()* 执行成功后的返回值。

- 返回值

目录流位置 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.19. ql_seekdir

该函数用于设置目录流读取位置。

- 函数原型

```
int ql_seekdir(QDIR *pdir, int offset)
```

- 参数

pdir:

[In] 目录句柄。即 *ql_opendir()* 执行成功后的返回值。

offset:

[In] 目录距离开头的偏移值。单位：字节。

- 返回值

实际设置目录流读取位置 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.20. ql_rewinddir

该函数用于设置目录流读取位置于目录开头。

- 函数原型

```
int ql_rewinddir(QDIR *pdir)
```

- 参数

pdir:

[In] 目录句柄。即 *ql_opendir()* 执行成功后的返回值。

- 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.21. ql_rmdir

该函数用于删除指定空目录。

- 函数原型

```
int ql_rmdir(const char *path)
```

- 参数

path:

[In] 目录名（带路径）。路径格式与 *ql_fopen()* 中的 *file_name* 一致。

- 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.22. ql_rename

该函数用于重命名文件。

- 函数原型

```
int ql_rename(const char *oldpath, const char *newpath)
```

- 参数

oldpath:

[In] 原文件名（带路径）。格式与 *ql_fopen()* 中 *file_name* 一致。

newpath:

[In] 新文件名（带路径）。格式与 *ql_fopen()* 中 *file_name* 一致。

- 返回值

详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.23. ql_fs_free_size

该函数用于通过文件（带路径）、目录路径或分区名，获取其所在分区的剩余空间大小。单位：字节。

- 函数原型

```
int64 ql_fs_free_size(const char *path)
```

- 参数

path:

[In] 文件（带路径）、目录路径或分区名。如 UFS:a.txt、UFS 或 SD。

- 返回值

分区剩余空间大小	函数执行成功
其他值	函数执行失败

3.3.24. ql_nvm_fwrite

该函数用于写入简单配置文件。

- 函数原型

```
int ql_nvm_fwrite(const char *config_file_name, void *buffer, size_t size, size_t num)
```

- 参数

config_file_name:

[In] 简单配置文件名。不可带路径。

buffer:

[In] 存储写入数据的内存地址。不可为空指针。

size:

[In] 需写入的每个元素大小。单位：字节。

num:

[In] 需写入的元素个数。最终写入数据大小为 $\text{size} \times \text{num}$ 。

- 返回值

写入数据大小	函数执行成功
其他值	函数执行失败；详见头文件中 <i>ql_file_errcode_e</i> 枚举相关信息。

3.3.25. ql_nvm_fread

该函数用于读取简单配置文件。

- 函数原型

```
int ql_nvm_fread(const char *config_file_name, void *buffer, size_t size, size_t num)
```

- 参数

config_file_name:

[In] 简单配置文件名。不可带路径。

buffer:

[In] 存储读取数据的内存地址。不可为空指针。

size:

[In] 需读取的每个元素大小。单位：字节。

num:

[In] 需读取的元素个数。最终读取数据大小为 $size \times num$ 。

- 返回值

读取数据大小 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.26. ql_sfs_setkey

该函数用于配置读写的 SFS 文件密钥。

- 函数原型

```
int ql_sfs_setkey(void *key, uint8_t len)
```

- 参数

key:

[In] 存储密钥的内存地址。不可为空指针。

len:

[In] 需设置的密钥的长度。范围：1~16；单位：字节。

- 返回值

详见头文件中 `ql_file_errcode_e` 枚举相关信息。

3.3.27. ql_fs_free_size_by_fd

该函数用于通过文件句柄获取文件所在分区的剩余空间大小。单位：字节。

- 函数原型

```
int64 ql_fs_free_size_by_fd(int fd)
```

- 参数

fd:

[In] 文件句柄。即 `ql_fopen()` 执行成功后的返回值。

- 返回值

分区剩余空间大小	函数执行成功
其他值	函数执行失败；详见头文件中 <code>ql_file_errcode_e</code> 枚举相关信息。

3.3.28. ql_fputc

该函数用于向文件中写入一个字符。

- 函数原型

```
int ql_fputc(int chr, int fd)
```

- 参数

chr:

[In] 写入的字符的 ASCII 值。

fd:

[In] 文件句柄。即 `ql_fopen()` 执行成功后的返回值。

- 返回值

写入字符的 ASCII 值	函数执行成功
其他值	函数执行失败；详见头文件中 <code>ql_file_errcode_e</code> 枚举相关信息。

3.3.29. ql_fgetc

该函数用于从文件中读取一个字符。

- 函数原型

```
int ql_fgetc(int fd)
```

- 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

- 返回值

读取字符的 ASCII 值 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.30. ql_ferror

该函数用于判断当前文件句柄是否有效。

- 函数原型

```
int ql_ferror(int fd)
```

- 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

- 返回值

0 当前文件句柄有效

其他值 无效输入；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.31. ql_fsync

该函数用于同步内存中已修改的文件数据到储存设备。

- 函数原型

```
int ql_fsync(int fd)
```

- 参数

fd:

[In] 文件句柄。即 *ql_fopen()* 执行成功后的返回值。

- 返回值

0 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.32. ql_cust_nvm_fwrite

该函数用于写入用户自定义简单配置文件。

- 函数原型

```
int ql_cust_nvm_fwrite(void *buffer, size_t size, size_t num)
```

- 参数

buffer:

[In] 存储写入数据的内存地址。不可为空指针。

size:

[In] 需写入的每个元素大小。单位：字节。

num:

[In] 需写入的元素个数。最终写入数据大小为 $size \times num$ 。

- 返回值

写入数据大小 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

3.3.33. ql_cust_nvm_fread

该函数用于读取用户自定义简单配置文件。

- 函数原型

```
int ql_cust_nvm_fread(void *buffer, size_t size, size_t num)
```

- 参数

buffer:

[In] 存储读取数据的内存地址。不可为空指针。

size:

[In] 需读取的每个元素大小。单位：字节。

num:

[In] 需读取的元素个数。最终读取数据大小为 $\text{size} \times \text{num}$ 。

- 返回值

读取数据大小 函数执行成功

其他值 函数执行失败；详见头文件中 *ql_file_errcode_e* 枚举相关信息。

4 示例

SDK 中提供文件系统代码示例文件 `ql_fs_demo.c`，路径为 `components\ql-application\fs`。用户可参考此文件实现相关功能。

5 附录 参考文档及术语缩写

表 3: 参考文档

文档名称
[1] Quectel_LTE_Standard(U)系列_QuecOpen_CSDK_快速开发指导
[2] Quectel_EC200D 系列_QuecOpen_CSDK_快速开发指导

表 4: 术语缩写

缩写	英文全称	中文全称
API	Application Programming Interface	应用程序接口
ASCII	American Standard Code for Information Interchange	美国信息交换标准码
RTOS	Real Time Operating System	实时操作系统
SDK	Software Development Kit	软件开发工具包
UFS	User File System	用户文件系统
EFS	External File System	外挂 Flash 文件系统
SD	Secure Digital Memory Card/SD card	安全数码卡
SFFS	Small-File-Oriented File System	面向小文件的文件系统
SFS	Secure File System	安全文件系统
SPI	Serial Peripheral Interface	串行外设接口